

GradeSense API Documentation

REST, WebSocket, governance, and typed client contracts

Curated, implementation-linked documentation. Code examples are selectable text, never screenshots.

Candidate

Name: Bhavuk Mahajan

Candidate ID: 20484629



1. API Contract and Conventions

Base API: FastAPI application with JSON request/response contracts. Validation errors return HTTP 422. Operational errors use structured error payloads. The OpenAPI UI is generated from the same Pydantic schemas used at runtime.

Concern	Contract
Content type	application/json
Identifiers	UUID strings
Timestamps	ISO 8601 UTC
Pagination	page, page_size and metadata
WebSocket events	type + timestamp + payload
Health	GET /health

2. System and Prediction Endpoints

Method	Endpoint	Purpose
GET	/health	Readiness and artifact availability
GET	/version	Application version
POST	/predict	Snapshot quality, risk and stabilization
POST	/recommend	Snapshot recommendation list
GET	/predictions	Persisted prediction history
GET	/recommendations	Persisted recommendation history
GET	/model/info	Active model metadata and metrics

Actual route implementation

```

router = APIRouter(tags=["intelligence"])

def get_intelligence_service(session: DatabaseSession) -> IntelligenceService:
    return IntelligenceService(get_settings(), session)

IntelligenceDependency = Annotated[IntelligenceService, Depends(get_intelligence_service)]

@router.post("/predict", response_model=PredictionResponse, summary="Predict transition quality")
def predict(process_input: ProcessInput, service: IntelligenceDependency) -> PredictionResponse:
    return service.predict(process_input)

@router.post(
    "/recommend",
    response_model=RecommendationResponse,
    summary="Generate model-evaluated corrective actions",
)
def recommend(
    process_input: ProcessInput, service: IntelligenceDependency
) -> RecommendationResponse:
    return service.recommend(process_input)

@router.get(
    "/correlations",
    response_model=CorrelationResponse,
    summary="Discover process correlations",
)
def correlations(
    service: IntelligenceDependency,
    limit: Annotated[int, Query(ge=1, le=50)] = 10,
) -> CorrelationResponse:
    return service.correlations(limit)

```

3. Prediction Request and Response

The request contains current/target grade plus validated process measurements. The response persists the prediction and includes confidence and explainability.

```
{
  "current_grade": "Kraft",
  "target_grade": "CopyPaper",
  "machine_speed": 880,
  "steam_pressure": 5.4,
  "dryer_temperature": 104,
  "moisture": 7.4,
  "basis_weight": 86,
  "caliper": 112,
  "pulp_consistency": 3.5,
  "stock_flow": 3400,
  "refining_energy": 160,
  "headbox_pressure": 3.8,
  "reel_tension": 5.2,
  "ambient_temperature": 30,
  "humidity": 72
}

{
  "prediction_id": "uuid",
  "quality_score": 86.1,
  "off_spec_probability": 0.243,
  "stabilization_time": 28.7,
  "confidence": 0.861,
  "explanation": {
    "summary": "Risk is driven by current process deviation.",
    "top_features": []
  }
}
```

4. Forecasting Endpoints

Method	Endpoint	Purpose
POST	/forecast	Persist a direct multi-horizon forecast
GET	/forecast/history	Paginated forecast history
GET	/forecast/{forecast_id}	Retrieve one forecast
POST	/forecast/simulate	Compare a constraint-valid candidate
GET	/relationships/discovery	Lag, nonlinear and interaction evidence

Forecast route

```

    ],
    explanation=row.explanation,
    created_at=row.created_at,
)

@router.post("/forecast", response_model=ForecastResponse)
def create_forecast(request: ForecastRequest, session: DatabaseSession) -> ForecastResponse:
    service = ForecastingService(get_settings())
    try:
        result = service.forecast(request)
    except FileNotFoundError as error:
        raise HTTPException(status_code=503, detail=str(error)) from error
    row = ForecastHistory(
        id=result.forecast_id,
        transition_id=result.transition_id,
        model_version=result.model_version,
        request_data=request.model_dump(mode="json"),
        trajectory=[point.model_dump(mode="json") for point in result.trajectory],
        specification=result.specification.model_dump(mode="json"),
        confidence=result.confidence,
        explanation=result.explanation,
        top_influencing_variables=[list(item) for item in result.top_influencing_variables],
    )
    session.add(row)
    crossing = result.specification
    if crossing.predicted_crossing_step and crossing.predicted_crossing_time:
        point = result.trajectory[crossing.predicted_crossing_step - 1]
        session.add(
            ForecastCrossingEvent(

```

5. Forecast Request and Response

```
{
  "transition_id": "DEMO-TR-00001",
  "history": [
    { "timestep": 1, "basis_weight": 76.15, "target_basis_weight": 86.0 }
  ],
  "horizon": 12
}

{
  "forecast_id": "uuid",
  "crossing_probability": 0.991,
  "crossing_step": 1,
  "maximum_deviation_percent": -10.74,
  "confidence": 0.281,
  "trajectory": [
    { "step": 1, "median": 76.4, "lower": 75.7, "upper": 77.1 }
  ]
}
```

6. Intervention and Outcome Endpoints

Method	Endpoint	Purpose
POST	/interventions/recommendations	Generate ranked forecast-backed actions
GET	/interventions/recommendations	Recommendation history
POST	/interventions/decisions	Accept, reject, modify, delay or apply
POST	/interventions/outcomes	Evaluate aligned future observations
GET	/interventions/effectiveness	Aggregate recommendation effectiveness

```
{
  "forecast_id": "uuid",
  "max_recommendations": 5
}

{
  "items": [{
    "rank": 1,
    "changes": [{"variable": "stock_flow", "suggested": 3280}],
    "metrics": {
      "crossing_probability_before": 0.81,
      "crossing_probability_after": 0.43,
      "estimated_improvement": 0.47
    },
    "constraint_validation": {"feasible": true}
  }]
}
```

7. Real-Time and Operational Endpoints

Method	Endpoint	Purpose
GET	/alerts	Alert history
POST	/alerts/{id}/acknowledge	Acknowledge alert
GET/POST	/feedback	Operator feedback
GET	/metrics/live	Latest sensor and inference state
GET	/metrics/rolling	Rolling windows
GET	/stream/status	Streaming state
WS	/ws/live	Live event channel
WS	/ws/dashboard	Dashboard event alias

WebSocket handler

```
@router.get("/drift", response_model=DriftResponse | None)
def drift() -> DriftResponse | None:
    return get_streaming_service().latest.drift

async def websocket_endpoint(websocket: WebSocket) -> None:
    service = get_streaming_service()
    await service.connections.connect(websocket)
    await service.connections.broadcast("system_status", service.status_response())
    try:
        while True:
            try:
                await websocket.receive_text()
            except WebSocketDisconnect:
                break
    finally:
        service.connections.disconnect(websocket)

router.add_api_websocket_route("/ws/live", websocket_endpoint)
router.add_api_websocket_route("/ws/dashboard", websocket_endpoint)
```


8. Model Governance and Administration

Group	Endpoints
Registry	/models, /models/active, register, promote, archive
Observability	/admin/metrics, /admin/health, /admin/system
Governance	/admin/audit, /admin/config
Exports	/admin/exports, /admin/export
Demo	POST /demo/seed

Promotion validates artifact existence, SHA-256, model version, metrics, feature schema and required inference interfaces. Active models cannot be archived without a promoted replacement.

9. Error Model and Client Integration

```
{
  "error": {
    "code": "validation_error",
    "message": "Request validation failed.",
    "details": [{"field": "steam_pressure", "reason": "outside safe range"}],
    "request_id": "uuid"
  }
}
```

Typed frontend request helper

```
RelationshipDiscovery,
DemoSeedResult,
} from './types'

const API_URL = (import.meta.env.VITE_API_URL ?? '').trim().replace(/\/$/, '')
if (!API_URL) {
  throw new Error('VITE_API_URL must be configured before building or starting GradeSense.')
}

export class ApiError extends Error {
  constructor(
    message: string,
    public status?: number,
    public details?: unknown,
  ) {
    super(message)
  }
}

async function request<T>(
  path: string,
  options?: RequestInit,
  timeoutMs = 30_000,
): Promise<T> {
  const controller = new AbortController()
  const timeout = window.setTimeout(() => controller.abort(), timeoutMs)
  try {
    const response = await fetch(`${API_URL}${path}`, {
      ...options,
      headers: { 'Content-Type': 'application/json', ...options?.headers },
      signal: controller.signal,
    })
    const payload = await response.json().catch(() => null)
    if (!response.ok) {
      const message =
        payload?.error?.message ||
        (typeof payload?.detail === 'string' ? payload.detail : undefined) ||
        payload?.detail?.[0]?.msg ||
        `Request failed (${response.status})`
      throw new ApiError(message, response.status, payload)
    }
    return payload as T
  }
}
```

Client code resolves REST and WebSocket origins from VITE_API_URL and fails fast when the environment variable is missing. This avoids silently sending production requests to an unintended origin.

10. API Verification Checklist

- Validate /health before demo traffic.
- Use the OpenAPI schema as the canonical request contract.
- Confirm UUID and timestamp serialization.
- Exercise 422 validation paths and structured error responses.
- Test WebSocket reconnect and event ordering.
- Verify decision and outcome persistence after recommendations.
- Inspect audit records for model promotion and runtime configuration changes.